CSE 4345: Big Data Analytics

Week 11, Part 1 — Next Steps: Emerging Trends, Ethics & Course Synthesis

Department of Computer Science & Engineering
Premier University, Chittagong

Semester 2026

Course & Lecture Information

Lecture Identity

Course:	CSE 4345
Week / Part:	11 of 11 | Part 1 of 2
CLO:	CLO4
PLO:	PLO(f)

CLO4

"Utilise the knowledge gained to solve big data problems and build a project applying big data tools and frameworks to real-world datasets"

Course Progression

Weeks 1–10: Foundations → storage → compute → query → pipelines → visualisation → real

Part 1 Covers

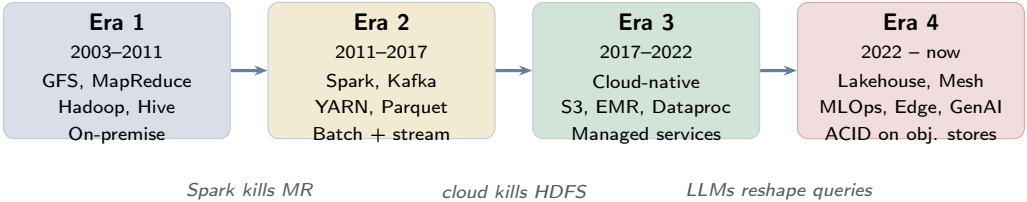
- The **technology evolution arc**: Hadoop on-premise → cloud-native → Lakehouse
- **Lakehouse architecture**: Delta Lake, Apache Iceberg, ACID on object stores; unified BI + ML
- **Data Mesh**: decentralised domain-oriented data ownership (Dehghani 2019)
- **MLOps**: operationalising ML at scale — MLflow, Airflow, Kubeflow, feature stores
- **Edge analytics**: processing at the IoT source — AWS Greengrass, Azure IoT Edge
- **Privacy-preserving analytics**: federated learning, differential privacy
- **Generative AI on big data**: Text-to-SQL,

Lecture Agenda

1. The Technology Evolution Arc
2. Lakehouse Architecture
3. Data Mesh
4. MLOps and Feature Stores
5. Edge Analytics and Cloud-Native Big Data
6. Privacy-Preserving Analytics
7. Course Synthesis and Career Pathways
8. Case Study: Apache Beam & Google Dataflow
9. Ivy League Alignment
10. Student Assessment Pool

The Technology Evolution Arc

How Big Data Technology Has Evolved: 2003 – 2025



Key Insight for Practitioners

Tools from Era 1 (HDFS, MapReduce) are still running in production at large enterprises. Technologies do not die when they are superseded — they *stratify* into legacy tiers. Understanding the full arc is essential for a Big Data Architect who must maintain old systems while migrating to new ones.

Lakehouse Architecture

Lakehouse: Unifying Data Lakes and Data Warehouses

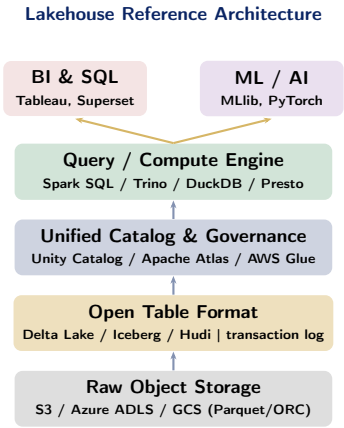
The Problem: Two Architectures, Two Pain Points

Data Lake (pure): Raw files on S3 / HDFS. Cheap storage, supports any format. But: **no ACID transactions** (concurrent writes corrupt data), **no schema enforcement** (data swamp risk), slow BI queries (no indexing).

Data Warehouse (Hive / Redshift): ACID, schema-on-write, fast BI. But: proprietary format locks in vendor, ML teams cannot access raw data, expensive to store all historical data.

Lakehouse (Zaharia et al., CIDR 2021):

- Open table format (**Delta Lake**, Apache **Iceberg**, Apache **Hudi**) sits on top of cheap object storage (S3 / Azure ADLS)
- Provides **ACID transactions**, **time travel** (query data as of any past point), **schema evolution**, and **audit**



Delta Lake and Apache Iceberg: ACID on Object Stores

How Delta Lake Achieves ACID

Delta Lake (Armbrust et al., VLDB 2020) stores a **transaction log** (a set of JSON files called the **delta log**) alongside the Parquet data files in S3:

1. Every write appends a new **commit entry** to the delta log listing: files added, files deleted, schema changes, metadata
2. Readers look up the latest committed state from the log before scanning data files — **snapshot isolation**
3. If two writers try to commit conflicting changes simultaneously, the second writer's optimistic concurrency check fails and it retries — **serialisable writes**
4. **Time travel:** Query data as of any previous snapshot: `VERSION AS OF 42` or `TIMESTAMP AS`

Data Lakehouse vs Alternatives

	Data Lake	**DW**	**Lakehouse**
ACID	No	Yes	**Yes**
Open format	Yes	No	**Yes**
BI queries	Slow	Fast	**Fast**
ML / raw	Yes	No	**Yes**
Storage cost	Low	High	**Low**
Schema enf.	No	Yes	**Yes**

Bangladesh context: bKash's transaction data (50 M transactions/day, 5+ years of

Data Mesh

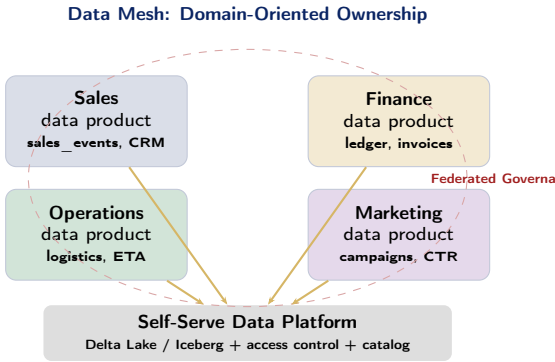
Data Mesh: Decentralised Domain-Oriented Data Ownership

Why Centralised Data Platforms Create Bottlenecks

In the traditional model, a **central data engineering team** ingests data from every business domain, builds pipelines, and publishes datasets. At scale, this creates:

- **Ownership mismatch:** the team that built the pipeline does not understand the domain; the domain team that understands the data cannot change the pipeline
- **Queue bottleneck:** 200 teams competing for 20 data engineers
- **Quality degradation:** nobody is accountable for downstream data correctness

Data Mesh (Zhamak Dehghani, 2019 / 2022) is a



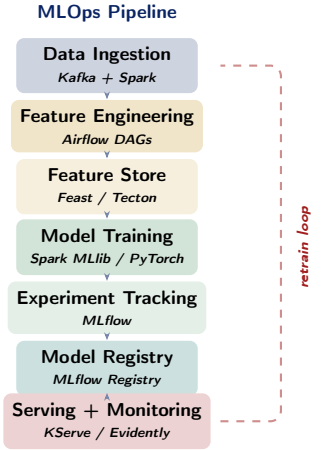
MLOps and Feature Stores

MLOps: From Experiment to Production ML

The Hidden Debt in ML Systems (Sculley et al., NeurIPS 2015)

"Only a small fraction of real-world ML systems is composed of the ML code. The required surrounding infrastructure is vast and complex." A trained model that lives only in a Jupyter notebook delivers zero business value. **MLOps** (ML Operations) is the discipline of reliably **deploying**, **monitoring**, and **retraining** models in production:

- **Data pipeline:** reproducible feature engineering (Apache Airflow DAGs)
- **Experiment tracking:** log hyperparameters, metrics, and artefacts (**MLflow**)
- **Feature store:** centralised repository of reusable features shared across teams; prevents



Edge Analytics and Cloud-Native Big Data

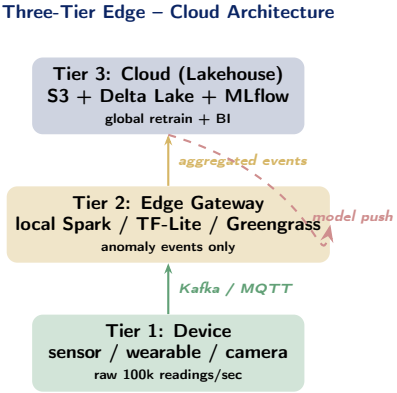
Edge Analytics: Pushing Computation to the Data Source

Why Edge?

The IoT data generation model (wearables, industrial sensors, CCTV cameras, RMG factory floor monitors) creates a **bandwidth and latency bottleneck**: transmitting every raw sensor reading to a central cloud cluster is impractical.

Edge analytics places lightweight compute *at or near* the source:

- **Tier 1 (device):** pre-processing, filtering, anomaly detection on microcontroller or FPGA (e.g., TensorFlow Lite on a wearable ECG patch)
- **Tier 2 (edge gateway):** local aggregation, windowed statistics, pre-trained model inference. Only summary events are forwarded to the cloud. *Example:* AWS Greengrass on a factory-floor server; Azure IoT Edge on an RMG weaving



Cloud-Native Big Data

On-premise Hadoop clusters are being replaced by

Privacy-Preserving Analytics

Federated Learning and Differential Privacy

Federated Learning: Train Without Sharing Data

In healthcare, finance, and government, **raw data cannot leave organisational boundaries** due to regulation (HIPAA, PDPA Bangladesh, GDPR). **Federated Learning** (McMahan et al., Google 2017) enables:

1. A **global model** is pushed to each participating hospital / bank
2. Each institution trains the model on its **local data** only
3. Only **model weight updates** (gradients) — not raw patient records — are sent to a central aggregation server
4. The server applies **FedAvg** (weighted average of

Differential Privacy: Provable Anonymisation

A mechanism $\mathcal{M}$ satisfies (ε, δ)-**differential privacy** if removing or adding any single individual's record changes the output distribution by at most a factor of e^{ε} (with probability $1 - \delta$).

In practice: Add **Laplace noise** (calibrated to sensitivity Δf and privacy budget ε) to aggregate query results before publishing:

$$\tilde{f}(D) = f(D) + \text{Lap}\left(\frac{\Delta f}{\varepsilon}\right)$$

Smaller $\varepsilon \Rightarrow$ more privacy, less accuracy. The privacy budget is "spent" with each query.

Real deployments:

- Apple: differential privacy for keyboard usage

Course Synthesis and Career Pathways

Course Synthesis: 11-Week CLO Coverage Map

Week	Core Topic	CLO	Key Tool / Concept
1	Digital data, 3Vs, CAP theorem, Batch vs Stream	CLO1	Hadoop ecosystem framing
2	Data integration challenges, ETL vs ELT	CLO2	HL7 FHIR, schema mapping
3	HDFS architecture, Hadoop ecosystem	CLO1	NameNode, DataNode, blocks
4	MapReduce, HBase, columnar storage	CLO1	Word count, column families
5	YARN, Parquet / ORC / Avro file formats	CLO3	Columnar encoding, predicate pushdown
6	Spark, RDDs, DataFrames, lazy evaluation	CLO3	DAG, narrow/wide dependencies
7	HiveQL, partitioning, bucket-	CLO3,4	SQL-on-Hadoop, directory pruning

Career Pathways: From CSE 4345 to Industry

Role	Skills from This Course	Additional Skills Needed
Data Engineer	Hadoop, Spark, Kafka, Hive, Sqoop; pipeline design	Python (pandas, PySpark), SQL, cloud (AWS/GCP), Airflow
Data Analyst	HiveQL, visualisation (Grafana, Tableau), SQL-on-Hadoop	Statistics, Power BI, Excel, business domain knowledge
Data Scientist	Spark MLlib, time series (Prophet), data pipelines	ML/DL (PyTorch / TF), statistics, A/B testing, feature engineering
Big Data Architect	Full-stack ecosystem design, Lambda/Kappa, Lakehouse	Cloud architecture (AWS Solutions Architect), security, cost modelling
ML Engineer	Spark, Kafka integration, batch forecasting	MLOps (MLflow, Kubeflow), Docker/K8s, model serving

Case Study: Apache Beam & Google Dataflow

Apache Beam: Eliminating the Lambda Architecture's Dual Code Path

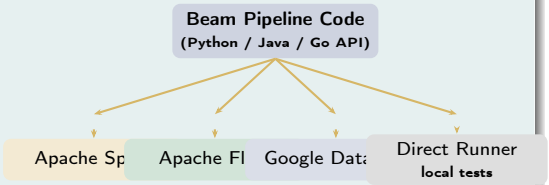
The Lambda Architecture's Fatal Flaw

The Lambda architecture (Week 8) solves batch–stream unification but at a cost: **you must maintain two codebases** — one MapReduce / Spark Batch job and one Storm / Spark Streaming job that implement identical business logic. When logic changes, both must be updated in sync. In practice, they drift and produce inconsistent results. **Akidau et al. (VLDB 2015)** introduced the **Dataflow Model** at Google, answering the question: *"Can a single programming model handle both batch and unbounded streaming data correctly?"*
The model introduces four primitives:

- **What** results are computed (aggregation, join)
- **Where** in event time results are computed (windowing)

Apache Beam: One Code, Multiple Runners

Apache Beam (open-source release of the Dataflow Model, 2016) lets you write one pipeline that runs on **multiple backends**:



A fraud detection pipeline written in Beam runs in batch mode (Spark) for historical replay and in streaming mode (Flink) in production — using

Ivy League Alignment

Emerging Trends in Top University Curricula

MIT 6.830 — Database Systems (Madden, Stonebraker)

MIT's database course dedicates its final module to **post-HDFS architectures**:

- The **Lakehouse vs. data warehouse debate** is a capstone seminar: students read Armbrust (2020) and debate whether Delta Lake renders Snowflake and Redshift obsolete
- **Federated query optimisation** is a graded problem set: students extend a query planner to push predicates across Delta Lake + Postgres + S3 JSON in a single query
- The **Dataflow Model** (Akidau 2015) is required reading, with a lecture devoted to the "What/Where/When/How" framework as the theoretical foundation for unified batch/stream

Stanford CS329S — ML Systems (Huyen)

Stanford's ML Systems course is the closest equivalent to our MLOps content:

- **Hidden Technical Debt** (Sculley 2015) is the first assigned paper; students map every component of the "debt taxonomy" onto a real production ML system
- **Feature store design** is a graded project: students implement a time-aware feature store (preventing future data leakage) using Feast on a Kafka–Spark pipeline
- The course argues that **data-centric AI** (improving data quality rather than model architecture) is the highest-leverage activity for production ML engineers —

Student Assessment Pool

Assessment—MCQ Set A

CLO4

Instructions

Select the single best answer. Correct answers **bolded** for instructor reference.

Q1. The Lakehouse architecture primarily combines:

a. MapReduce batch processing and Apache Hive SQL
b. **Data lake flexible storage with data warehouse reliability (ACID transactions and schema enforcement)**
c. Streaming ingestion and edge analytics
d. Cloud object storage with in-memory graph processing

Q2. Apache Iceberg and Delta Lake address which fundamental limitation of traditional data lakes?

a. Poor compression ratios on object storage
b. Lack of SQL query support
c. **Absence of ACID transactions and schema enforcement, leading to unreliable concurrent writes**
d. Inability to store unstructured data such as images or documents

Instructor Notes

Q1: The Lakehouse does not replace streaming (c) or add graph processing (d) — it unifies lake

Assessment—MCQ Set B

CLO4

Q3. The Data Mesh concept was proposed primarily to solve:

a) Storage cost problems in HDFS at petabyte scale
b) Streaming latency issues in Kafka consumer groups
c) **Bottlenecks caused by centralised data teams owning and serving all organisational data pipelines**
d) Security vulnerabilities in Hadoop's authentication model

Q4. Federated Learning is primarily used for:

a) Distributed SQL queries across Hive metastore partitions
b) Real-time event stream processing with sub-second latency
c) **Training machine learning models on distributed data without transferring raw data to a central location, preserving privacy**
d) Graph analytics and community detection on social networks

Instructor Notes

Q3: Data Mesh is an *organisational* architecture (not a storage or latency solution) that transfers data ownership from a central team to domain teams. **Q4:** The defining property of Federated Learning is that **raw data never moves** — only model gradient updates are aggregated centrally via FedAvg. This is the mechanism that makes it privacy-preserving for healthcare and banking.

Assessment—True / False

CLO4

Instructions

State True or False and provide a **one-sentence justification** for full marks.

Statement	Answer
1. Apache Beam allows the same pipeline code to run on both Apache Spark and Apache Flink as backend runners.	**True**
2. Cloud-managed services such as AWS EMR require the user to provision and manage Hadoop cluster hardware manually.	**False**
3. Differential privacy guarantees that individual records cannot be inferred from aggregate query results, at the cost of some accuracy.	**True**
4. The Data Mesh approach centralises all data ownership in a single platform team to improve governance and quality.	**False**

Assessment—Descriptive Questions *CLO4*

Instructions

Answer each question in **100–150 words**.

D1. Explain the **Lakehouse architecture**. What specific problems does it solve that neither a pure data lake nor a pure data warehouse could solve individually? Name one open-source technology that implements the Lakehouse concept and describe its core mechanism.

D2. What is **federated learning**? Give a specific healthcare example where federated learning is preferable to centralising all patient data on a single cluster. What are two **limitations** of federated learning that a system architect must account for?

D3. Reflect on the 11-week course: explain how **HDFS** $\rightarrow$ **MapReduce** $\rightarrow$ **Hive** $\rightarrow$ **Spark** $\rightarrow$ **Kafka** $\rightarrow$ **Lakehouse** forms a coherent and chronologically motivated technology stack. What was the driving force behind each transition?

Assessment—Capstone Analytical Questions *CLO4*

Instructions

Answer in **250–350 words** with full end-to-end pipeline reasoning and explicit tool justification.

A1. National Mobile Payment Platform Design:
Design a complete big data analytics platform for a nationwide mobile payment company in Bangladesh (like bKash) handling 50 million transactions per day. Your design must include: (a) data ingestion layer — how transactions enter the system; (b) storage architecture — HDFS or Lakehouse on cloud; (c) batch analytics layer — for monthly regulatory reports; (d) real-time fraud detection layer — achieving sub-2-second latency; (e) visualisation dashboard. Name every tool from this course you would use and justify each choice against a specific alternative.

A2. Ethics — Social Media Mental Health Prediction:
A researcher uses 5 years of Facebook posts from 2 million Bangladeshi users (scraped without explicit consent) to train a model predicting depression with 90% accuracy. (a) What are the privacy and ethical concerns? (b) How could **federated learning** or **differential privacy** mitigate these concerns? (c) Should this model be deployed in a hospital setting? Justify your answer with reference to HIPAA/PDPA principles and the risk of algorithmic bias.

Textbook & Reading References

Primary Textbooks for Week 11

- **[SA]** Acharya & Chellappan. *Big Data and Analytics*, Wiley, 2015. **Ch. 1, 12**
  - Ch. 12: Future directions in big data; cloud integration; privacy
- **[TE]** Erl, Khattak & Buhler. *Big Data Fundamentals*, Prentice Hall, 2016. **Ch. 14**
  - Ch. 14: Big data governance, security, and emerging patterns
- **[MMS]** Mayer-Schönberger & Cukier. *Big Data: A Revolution*, Houghton Mifflin Harcourt, 2013. **Epilogue**
  - Risks and responsibilities of big data; datafication and its social consequences

Seminal Papers (covered in Week 11, Part 2)

- Armbrust, M. et al. (2020). **Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores.** *VLDB 2020.*
- Zaharia, M. et al. (2021). **Lakehouse: A New Generation of Open Platforms That Unify Data Warehousing and Advanced Analytics.** *CIDR 2021.*
- Akidau, T. et al. (2015). **The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing.** *VLDB 2015.*

Congratulations — You Have Completed CSE 4345

Week 11, Part 1 | Big Data Analytics

"The world is now awash in data and we can see consumers in a lot clearer ways."

— Max Levchin, Co-founder of PayPal

Week 11, Part 2

Seminal Papers: Armbrust 2020 (Delta Lake) · Zaharia 2021 (Lakehouse) · Akidau 2015 (Dataflow Model)

Case Study: Databricks Lakehouse in production — how the company that built Spark now bets the company on Delta Lake

Capstone Project Reminder

Final Project + Exhibition is **10%** of your grade.

Apply one or more tools from this course to a **Bangladesh-context dataset**. Present your pipeline, results, and lessons learned.

Reading before Part 2: Armbrust et al. (2020) — open access on VLDB website.